

VILNIUS UNIVERSITY
FACULTY OF MATHEMATICS AND INFORMATICS
INSTITUTE OF COMPUTER SCIENCE
«DEPARTMENT OF COMPUTATIONAL AND DATA MODELING» OR
«CYBERSECURITY LABORATORY»

Semester Project

Fixing the Mathematical Foundations of the Lietuva Travel Category-Theoretic Engine

Corrección de los fundamentos matemáticos del motor categórico de Lietuva Travel

Done by:
Adrián Guinea Pérez signature

Supervisor:
Prof. I. Grinis

Vilnius
2026

Contents

Sutartinis terminų žodynas **3**

Abstract **4**

Santrauka **5**

Introduction **6**

1 Analysis of the Three Reported Defects **7**

1.1 Option Monad: `pure` Does Not Satisfy the Right-Identity Law 7

1.2 Monoid: `TripMetrics::append` Is Not Associative 7

1.3 Free Category $F(G)$: Composition Failures Were Swallowed 8

1.4 Regression Coverage 9

Conclusions and Recommendations **10**

Future Work **11**

Declaration of Artificial Intelligence **12**

References **13**

Sutartinis terminų žodynas

Category Theory, Monad, Monoid, Functor, Free Category, Natural Transformation, PHP 8.1

Abstract

This report documents the correction of three mathematical defects identified by Prof. I. Grinis in the *Lietuva Travel* categorical travel-planning engine (PHP 8.1+). Although the project's self-reported test suite claimed 15/15 category-theoretic laws verified, three real violations survived: (1) the Option monad's pure constructor conflated the falsy value false with absence of a value, breaking the monad's right-identity law for $\mathrm{Some}(\mathrm{false})$; (2) the $\mathrm{TripMetrics}$ monoid rounded its cost to two decimals after every $\mathrm{append}()$, which made the monoid's associativity law fail for sub-cent amounts; and (3) $\mathrm{Itinerary{::}advance}()$ silently discarded a failed path composition in the induced free category $F(G)$, letting an invalid itinerary succeed instead of failing. Each defect is analysed at its root cause, fixed with the smallest change that restores the law, and covered by a new regression assertion added to the existing law-verification suite, which now reports 18/18 laws passing.

Santrauka

Corrección de los fundamentos matemáticos del motor categórico de Lietuva Travel

Este informe documenta la corrección de tres defectos matemáticos señalados por el Prof. I. Grinis en el motor de planificación de viajes *Lietuva Travel* (PHP 8.1+), basado en teoría de categorías. Aunque la batería de pruebas del propio proyecto afirmaba verificar 15/15 leyes categóricas, subsistían tres violaciones reales: (1) el constructor pure de la mónada Option confundía el valor falso false con la ausencia de valor, rompiendo la ley de identidad derecha para $\mathrm{Some}(\mathrm{false})$; (2) el monoide $\mathrm{TripMetrics}$ redondeaba el coste a dos decimales tras cada $\mathrm{append}()$, lo que anulaba la asociatividad del monoide para cantidades inferiores al céntimo; y (3) $\mathrm{Itinerary{::}advance}()$ descartaba en silencio una composición de caminos fallida en la categoría libre inducida $F(G)$, dejando que un itinerario inválido se diera por válido en lugar de fallar. Cada defecto se analiza en su causa raíz, se corrige con el menor cambio posible que restaura la ley, y queda cubierto por una nueva aserción de regresión añadida a la batería de verificación existente, que ahora reporta 18/18 leyes superadas.

Introduction

Lietuva Travel is a PHP 8.1+ travel-planning application whose core logic (src/CategoryTheory.php) is explicitly built on category-theoretic structures, following the constructions of Barr and Wells [1]: a Functor/Monad interface pair in the style of Moggi's and Wadler's monadic treatment of computation [2, 3], the Option and Either monads, the RouteList functor with a natural transformation into Option, the TripMetrics monoid for associative cost aggregation, and a directed graph whose paths are lifted into a free category $F(G)$ (Prof. I. Grinis' model), where edge composition is interpreted as Kleisli composition in the Either monad.

The project ships its own formal verification suite (tests/category_laws_test.php), which prints a "15/15 laws verified" badge. Prof. Grinis reviewed the submission and reported, by e-mail, three concrete counter-examples that the existing suite failed to catch:

1. **Option monad law violation.** Option::pure(false) returned None instead of Some(false), because the implementation treated false as equivalent to null.

2. **Monoid associativity violation.** For $a = 0.004$, $b = 0.004$, $c = 0.005$ (euros), TripMetrics produced $(a \oplus b) \oplus c = 0.02$ but $a \oplus (b \oplus c) = 0.01$.

3. **Swallowed composition failure.** In Itinerary::advance(), a non-composable graph edge (target of the current path $\neq$ source of the next edge) was silently ignored instead of failing the itinerary computation.

This report explains, for each issue, why the existing law was actually violated despite the suite reporting success, the minimal code change that restores the law, and the new regression check added so the suite can no longer miss it. The fixes are implemented directly in src/CategoryTheory.php and, where relevant, in the one call site in index.php that depended on the old (incorrect) behaviour.

1 Analysis of the Three Reported Defects

1.1 Option Monad: pure Does Not Satisfy the Right-Identity Law

The Option monad is expected to satisfy the monad laws for *every* value its type parameter can hold, including bool. The original implementation of Option::pure was:

Listing 1. Original Option::pure (src/CategoryTheory.php)

```
1 public static function pure(mixed $value): self {
2     return ($value === null || $value === false)
3         ? new None()
4         : new Some($value);
5 }
```

Taking $m = \texttt{Some(false)}$ and $f = \text{Option::pure}$, the right-identity law $m \gg\!= \text{pure} \equiv m$ requires $m \gg\!= f$ to equal Some(false). Instead it produced None, because pure conflated the domain value false with the *absence* of a value — exactly the distinction the Option monad exists to preserve. The root cause is that null (absence) and false (a legitimate boolean payload) are not the same value, yet the guard treated them identically.

Fix. Only null denotes absence:

Listing 2. Fixed Option::pure

```
1 public static function pure(mixed $value): self {
2     return ($value === null) ? new None() : new Some($value);
3 }
```

One caller, fetchLiveJsonOption() in index.php, had silently relied on the old double meaning: file_get_contents() returns the sentinel false on failure, and the code built an Option from it via Option::pure($raw). Since pure no longer treats false as absence, that call site was changed to test the sentinel explicitly instead of overloading a general-purpose constructor with a local, API-specific meaning:

Listing 3. index.php – explicit sentinel check instead of overloaded pure()

```
1 $raw = @file_get_contents($url, false, $ctx);
2 return ($raw === false ? new None() : new Some($raw))
3     ->fmap(fn(string $json) => json_decode($json, true));
```

1.2 Monoid: TripMetrics::append Is Not Associative

TripMetrics is declared a Monoid, which requires $(a \oplus b) \oplus c = a \oplus (b \oplus c)$ for all values, exactly. The original append rounded the running total to two decimals after *every* addition:

Listing 4. Original TripMetrics::append

```
1 public function append(Monoid $other): self {
2     return new self(
3         round($this->costEur + $other->costEur, 2),
4         ...
5     );
6 }
```

For $a = 0.004$, $b = 0.004$, $c = 0.005$ (Table 1), rounding after each step amplifies the order in which values are combined into a visible difference: grouping $(a \oplus b)$ first rounds $0.008 \to 0.01$, then $0.01 + 0.005 \to 0.02$; grouping $(b \oplus c)$ first rounds $0.009 \to 0.01$, then $a + 0.01 \to 0.01$. The two totals disagree, so the monoid is not actually associative — the repeated rounding is a lossy, order-dependent operation disguised as a pure sum.

Table 1. Associativity counter-example reported by Prof. Grinis

Grouping	Original (buggy)	Fixed
$(a \oplus b) \oplus c$	0.02	0.01
$a \oplus (b \oplus c)$	0.01	0.01

Fix. Round to whole cents exactly once, at construction, and keep that integer as the value actually added:

Listing 5. Fixed TripMetrics – integer cents as the additive carrier

```
public readonly float $costEur;
public readonly int $costCents;

public function __construct(float $costEur, ...) {
    $this->costCents = (int) round($costEur * 100);
    $this->costEur = $this->costCents / 100;
}

public function append(Monoid $other): self {
    return new self(
        ($this->costCents + $other->costCents) / 100,
        ...
    );
}
```

Integer addition is exactly associative, so no grouping of append calls can disagree; the public costEur float is only ever a deterministic function of the accumulated integer cents, never an input to further rounding.

1.3 Free Category $F(G)$: Composition Failures Were Swallowed

In the free category $F(G)$ induced by the travel graph, FreeCategoryPath::compose already returns an Either: Right on success, Left when the target vertex of the current path does not match the source vertex of the next edge (the paths do not form a connected walk). Itinerary::advance(), however, used that result incorrectly:

Listing 6. Original Itinerary::advance – Left is discarded

```
$composed = $newPath->compose(FreeCategoryPath::fromEdge($graphEdge));
if ($composed->isRight()) {
    $newPath = $composed->getValue();
}
// execution continues either way
```

When composition fails, the if is simply not entered and $newPath keeps its previous value; advance() still returns a brand-new, successful Itinerary. The categorical error is computed correctly

and then thrown away, so an itinerary whose graph path is not actually a connected walk in $F(G)$ is reported as valid.

Fix. Propagate the failure instead of absorbing it:

Listing 7. Fixed Itinerary::advance – Left fails the computation

```
1 $composed = $newPath->compose(FreeCategoryPath::fromEdge($graphEdge));
2 if (!$composed->isRight()) {
3     $err = $composed->getValue();
4     throw new RuntimeException("{$err['title']}: {$err['msg']}");
5 }
6 $newPath = $composed->getValue();
```

A non-composable edge is an internal invariant violation of the step-building pipeline (every edge is constructed by the application itself from the previous step's state), not a user-facing validation outcome like a late flight or an over-budget trip, so failing fast with an exception is the appropriate signal, consistent with how every other Left produced inside this pipeline is already treated as a short-circuit.

1.4 Regression Coverage

Each fix above is paired with one new assertion in tests/category_laws_test.php, reproducing Prof. Grinis' exact examples: Some(false) round-tripping through bind(Option::pure); the $a = 0.004$, $b = 0.004$, $c = 0.005$ monoid grouping; and a two-step Itinerary::advance() call with a deliberately mismatched edge, asserting that it throws. Running php tests/category_laws_test.php against the corrected code reports **18/18** laws verified (up from the previous 15/15, which had missed all three cases above).

Conclusions and Recommendations

All three defects reported by Prof. I. Grinis were root-caused and fixed with minimal, localised changes to src/CategoryTheory.php (and one dependent call site in index.php):

- Option::pure now treats only null as absence of value, so Some(false) round-trips correctly through bind(Option::pure), restoring the monad's right-identity law. The single call site that relied on the old behaviour (fetchLiveJsonOption, which used the false sentinel returned by file_get_contents on failure) was updated to test for that sentinel explicitly instead of overloading Option::pure with two meanings.
- TripMetrics now rounds to whole cents exactly once, at construction time, and stores that integer value (costCents). append() adds integers, which is exactly associative, instead of adding floats and re-rounding to two decimals after every step, which was not. The public float interface (costEur) is unchanged, so no other call site required modification.
- Itinerary::advance() now throws a RuntimeException carrying the free category's own Left error message when a graph edge does not compose with the current induced path, instead of silently keeping the previous (shorter, incorrect) path and returning a successful itinerary.

Each fix is accompanied by a new assertion in tests/category_laws_test.php, reproducing the exact counter-example supplied by Prof. Grinis (including the $a = 0.004$, $b = 0.004$, $c = 0.005$ monoid case). The suite, previously reporting "15/15", now reports **18/18** laws verified, and the three new assertions fail against the old code and pass against the fixed code, confirming both the regressions and the fixes.

Future Work

Possible follow-ups include replacing the remaining ad hoc $\mathrm{InvalidArgumentException}$ checks in $\mathrm{Monad::bind}$ implementations with static analysis guarantees, and extending the law suite with randomised (property-based) inputs rather than fixed examples, so similar edge cases are found automatically rather than by manual review.

Declaration of Artificial Intelligence

Claude Code (Anthropic), an AI coding assistant, was used under the author's direction to: (i) locate the exact lines responsible for each of the three defects reported by Prof. Grinis; (ii) implement the corresponding fixes in src/CategoryTheory.php and index.php; (iii) add the three regression assertions to tests/category_laws_test.php; (iv) run the test suite (php tests/category_laws_test.php) to confirm 18/18 laws pass; and (v) draft this LaTeX report from the official department template. All changes were reviewed by the author before submission. No AI-generated content was included without verification against the actual code and test output.

References

[1] Michael Barr and Charles Wells. *Category Theory for Computing Science*. Prentice Hall, New York, 1990.

[2] Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.

[3] Philip Wadler. Monads for functional programming. *Lecture Notes in Computer Science*, 925:24–52, 1992.